

13

Text classification and Naive Bayes

STANDING QUERY

Thus far, this book has mainly discussed the process of *ad hoc retrieval*, where users have transient information needs that they try to address by posing one or more queries to a search engine. However, many users have ongoing information needs. For example, you might need to track developments in *multicore computer chips*. One way of doing this is to issue the query `multicore AND computer AND chip` against an index of recent newswire articles each morning. In this and the following two chapters we examine the question: How can this repetitive task be automated? To this end, many systems support *standing queries*. A standing query is like any other query except that it is periodically executed on a collection to which new documents are incrementally added over time.

If your standing query is just `multicore AND computer AND chip`, you will tend to miss many relevant new articles which use other terms such as *multicore processors*. To achieve good recall, standing queries thus have to be refined over time and can gradually become quite complex. In this example, using a Boolean search engine with stemming, you might end up with a query like `(multicore OR multi-core) AND (chip OR processor OR microprocessor)`.

CLASSIFICATION

To capture the generality and scope of the problem space to which standing queries belong, we now introduce the general notion of a *classification* problem. Given a set of *classes*, we seek to determine which class(es) a given object belongs to. In the example, the standing query serves to divide new newswire articles into the two classes: documents about multicore computer chips and documents not about multicore computer chips. We refer to this as *two-class classification*. Classification using standing queries is also called *routing* or *filtering* and will be discussed further in Section 15.3.1 (page 335).

ROUTING

FILTERING

TEXT CLASSIFICATION

A class need not be as narrowly focused as the standing query *multicore computer chips*. Often, a class is a more general subject area like *China* or *coffee*. Such more general classes are usually referred to as *topics*, and the classification task is then called *text classification*, *text categorization*, *topic classification*, or *topic spotting*. An example for *China* appears in Figure 13.1. Standing queries and topics differ in their degree of specificity, but the methods for

solving routing, filtering, and text classification are essentially the same. We therefore include routing and filtering under the rubric of text classification in this and the following chapters.

The notion of classification is very general and has many applications within and beyond information retrieval (IR). For instance, in computer vision, a classifier may be used to divide images into classes such as *landscape*, *portrait*, and *neither*. We focus here on examples from information retrieval such as:

- Several of the preprocessing steps necessary for indexing as discussed in Chapter 2: detecting a document's encoding (ASCII, Unicode UTF-8 etc; page 20); word segmentation (Is the white space between two letters a word boundary or not? page 24) ; truecasing (page 30); and identifying the language of a document (page 46).
- The automatic detection of spam pages (which then are not included in the search engine index).
- The automatic detection of sexually explicit content (which is included in search results only if the user turns an option such as SafeSearch off).

SENTIMENT DETECTION

- *Sentiment detection* or the automatic classification of a movie or product review as positive or negative. An example application is a user searching for negative reviews before buying a camera to make sure it has no undesirable features or quality problems.

EMAIL SORTING

- *Personal email sorting*. A user may have folders like *talk announcements*, *electronic bills*, *email from family and friends*, and so on, and may want a classifier to classify each incoming email and automatically move it to the appropriate folder. It is easier to find messages in sorted folders than in a very large inbox. The most common case of this application is a spam folder that holds all suspected spam messages.

VERTICAL SEARCH ENGINE

- Topic-specific or *vertical search*. *Vertical search engines* restrict searches to a particular topic. For example, the query computer science on a vertical search engine for the topic *China* will return a list of Chinese computer science departments with higher precision and recall than the query computer science China on a general purpose search engine. This is because the vertical search engine does not include web pages in its index that contain the term china in a different sense (e.g., referring to a hard white ceramic), but does include relevant pages even if they do not explicitly mention the term China.
- Finally, the ranking function in ad hoc information retrieval can also be based on a document classifier as we will explain in Section 15.4 (page 341).

This list shows the general importance of classification in IR. Most retrieval systems today contain multiple components that use some form of classifier. The classification task we will use as an example in this book is text classification.

RULES IN TEXT CLASSIFICATION

A computer is not essential for classification. Many classification tasks have traditionally been solved manually. Books in a library are assigned Library of Congress categories by a librarian. But manual classification is expensive to scale. The *multicore computer chips* example illustrates one alternative approach: classification by the use of standing queries – which can be thought of as *rules* – most commonly written by hand. As in our example (multicore OR multi-core) AND (chip OR processor OR microprocessor), rules are sometimes equivalent to Boolean expressions.

A rule captures a certain combination of keywords that indicates a class. Hand-coded rules have good scaling properties, but creating and maintaining them over time is labor intensive. A technically skilled person (e.g., a domain expert who is good at writing regular expressions) can create rule sets that will rival or exceed the accuracy of the automatically generated classifiers we will discuss shortly; however, it can be hard to find someone with this specialized skill.

STATISTICAL TEXT CLASSIFICATION

Apart from manual classification and hand-crafted rules, there is a third approach to text classification, namely, machine learning-based text classification. It is the approach that we focus on in the next several chapters. In machine learning, the set of rules or, more generally, the decision criterion of the text classifier, is learned automatically from training data. This approach is also called *statistical text classification* if the learning method is statistical. In statistical text classification, we require a number of good example documents (or training documents) for each class. The need for manual classification is not eliminated because the training documents come from a person who has labeled them – where *labeling* refers to the process of annotating each document with its class. But labeling is arguably an easier task than writing rules. Almost anybody can look at a document and decide whether or not it is related to China. Sometimes such labeling is already implicitly part of an existing workflow. For instance, you may go through the news articles returned by a standing query each morning and give relevance feedback (cf. Chapter 9) by moving the relevant articles to a special folder like *multicore-processors*.

LABELING

We begin this chapter with a general introduction to the text classification problem including a formal definition (Section 13.1); we then cover Naive Bayes, a particularly simple and effective classification method (Sections 13.2–13.4). All of the classification algorithms we study represent documents in high-dimensional spaces. To improve the efficiency of these algorithms, it is generally desirable to reduce the dimensionality of these spaces; to this end, a technique known as *feature selection* is commonly applied in text clas-

sification as discussed in Section 13.5. Section 13.6 covers evaluation of text classification. In the following chapters, Chapters 14 and 15, we look at two other families of classification methods, vector space classifiers and support vector machines.

13.1 The text classification problem

DOCUMENT SPACE
CLASS
TRAINING SET

In text classification, we are given a description $d \in \mathbb{X}$ of a document, where $\mathbb{X}$ is the *document space*; and a fixed set of *classes* $\mathbb{C} = \{c_1, c_2, \dots, c_J\}$. Classes are also called *categories* or *labels*. Typically, the document space $\mathbb{X}$ is some type of high-dimensional space, and the classes are human defined for the needs of an application, as in the examples *China* and *documents that talk about multicore computer chips* above. We are given a *training set* $\mathbb{D}$ of labeled documents $\langle d, c \rangle$, where $\langle d, c \rangle \in \mathbb{X} \times \mathbb{C}$. For example:

$$\langle d, c \rangle = \langle \text{Beijing joins the World Trade Organization}, \text{China} \rangle$$

for the one-sentence document *Beijing joins the World Trade Organization* and the class (or label) *China*.

LEARNING METHOD
CLASSIFIER

Using a *learning method* or *learning algorithm*, we then wish to learn a classifier or *classification function* γ that maps documents to classes:

$$(13.1) \quad \gamma : \mathbb{X} \rightarrow \mathbb{C}$$

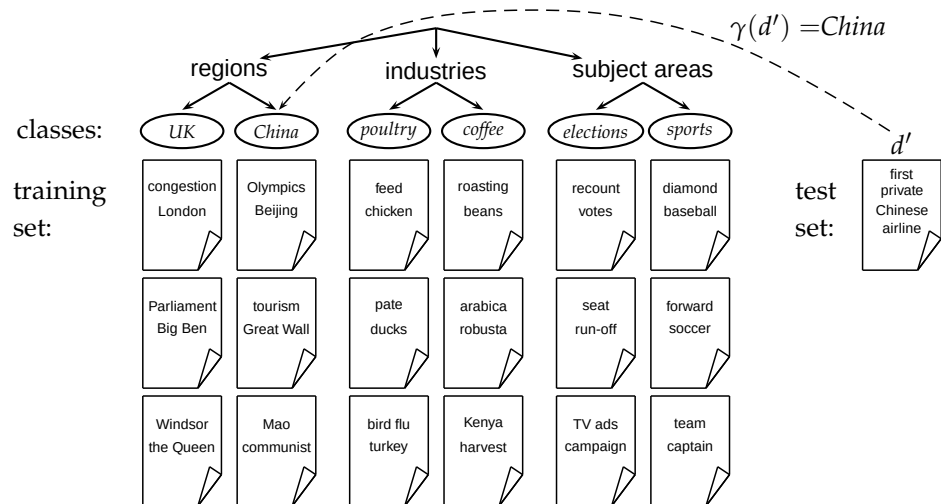
SUPERVISED LEARNING

This type of learning is called *supervised learning* because a supervisor (the human who defines the classes and labels training documents) serves as a teacher directing the learning process. We denote the supervised learning method by Γ and write $\Gamma(\mathbb{D}) = \gamma$. The learning method Γ takes the training set $\mathbb{D}$ as input and returns the learned classification function γ .

Most names for learning methods Γ are also used for classifiers γ . We talk about the Naive Bayes (NB) *learning method* Γ when we say that “Naive Bayes is robust,” meaning that it can be applied to many different learning problems and is unlikely to produce classifiers that fail catastrophically. But when we say that “Naive Bayes had an error rate of 20%,” we are describing an experiment in which a particular NB *classifier* γ (which was produced by the NB learning method) had a 20% error rate in an application.

TEST SET

Figure 13.1 shows an example of text classification from the Reuters-RCV1 collection, introduced in Section 4.2, page 69. There are six classes (*UK*, *China*, ..., *sports*), each with three training documents. We show a few mnemonic words for each document’s content. The training set provides some typical examples for each class, so that we can learn the classification function γ . Once we have learned γ , we can apply it to the *test set* (or *test data*), for example, the new document *first private Chinese airline* whose class is unknown.



► **Figure 13.1** Classes, training set, and test set in text classification .

In Figure 13.1, the classification function assigns the new document to class $\gamma(d) = \text{China}$, which is the correct assignment.

The classes in text classification often have some interesting structure such as the hierarchy in Figure 13.1. There are two instances each of region categories, industry categories, and subject area categories. A hierarchy can be an important aid in solving a classification problem; see Section 15.3.2 for further discussion. Until then, we will make the assumption in the text classification chapters that the classes form a set with no subset relationships between them.

Definition (13.1) stipulates that a document is a member of exactly one class. This is not the most appropriate model for the hierarchy in Figure 13.1. For instance, a document about the 2008 Olympics should be a member of two classes: the *China* class and the *sports* class. This type of classification problem is referred to as an *any-of* problem and we will return to it in Section 14.5 (page 306). For the time being, we only consider *one-of* problems where a document is a member of exactly one class.

Our goal in text classification is high accuracy on test data or *new data* – for example, the newswire articles that we will encounter tomorrow morning in the multicore chip example. It is easy to achieve high accuracy on the training set (e.g., we can simply memorize the labels). But high accuracy on the training set in general does not mean that the classifier will work well on

new data in an application. When we use the training set to learn a classifier for test data, we make the assumption that training data and test data are similar or from *the same distribution*. We defer a precise definition of this notion to Section 14.6 (page 308).

13.2 Naive Bayes text classification

MULTINOMIAL NAIVE
BAYES

The first supervised learning method we introduce is the *multinomial Naive Bayes* or *multinomial NB* model, a probabilistic learning method. The probability of a document d being in class c is computed as

$$(13.2) \quad P(c|d) \propto P(c) \prod_{1 \leq k \leq n_d} P(t_k|c)$$

where $P(t_k|c)$ is the conditional probability of term t_k occurring in a document of class c .¹ We interpret $P(t_k|c)$ as a measure of how much evidence t_k contributes that c is the correct class. $P(c)$ is the prior probability of a document occurring in class c . If a document's terms do not provide clear evidence for one class versus another, we choose the one that has a higher prior probability. $\langle t_1, t_2, \dots, t_{n_d} \rangle$ are the tokens in d that are part of the vocabulary we use for classification and n_d is the number of such tokens in d . For example, $\langle t_1, t_2, \dots, t_{n_d} \rangle$ for the one-sentence document *Beijing and Taipei join the WTO* might be $\langle \text{Beijing, Taipei, join, WTO} \rangle$, with $n_d = 4$, if we treat the terms and the as stop words.

MAXIMUM A
POSTERIORI CLASS

In text classification, our goal is to find the *best* class for the document. The best class in NB classification is the most likely or *maximum a posteriori* (MAP) class c_{map} :

$$(13.3) \quad c_{\text{map}} = \arg \max_{c \in C} \hat{P}(c|d) = \arg \max_{c \in C} \hat{P}(c) \prod_{1 \leq k \leq n_d} \hat{P}(t_k|c).$$

We write $\hat{P}$ for P because we do not know the true values of the parameters $P(c)$ and $P(t_k|c)$, but estimate them from the training set as we will see in a moment.

In Equation (13.3), many conditional probabilities are multiplied, one for each position $1 \leq k \leq n_d$. This can result in a floating point underflow. It is therefore better to perform the computation by adding logarithms of probabilities instead of multiplying probabilities. The class with the highest log probability score is still the most probable; $\log(xy) = \log(x) + \log(y)$ and the logarithm function is monotonic. Hence, the maximization that is

1. We will explain in the next section why $P(c|d)$ is proportional to $(\propto)$, not equal to the quantity on the right.

actually done in most implementations of NB is:

$$(13.4) \quad c_{\text{map}} = \arg \max_{c \in \mathbb{C}} [\log \hat{P}(c) + \sum_{1 \leq k \leq n_d} \log \hat{P}(t_k|c)].$$

Equation (13.4) has a simple interpretation. Each conditional parameter $\log \hat{P}(t_k|c)$ is a weight that indicates how good an indicator t_k is for c . Similarly, the prior $\log \hat{P}(c)$ is a weight that indicates the relative frequency of c . More frequent classes are more likely to be the correct class than infrequent classes. The sum of log prior and term weights is then a measure of how much evidence there is for the document being in the class, and Equation (13.4) selects the class for which we have the most evidence.

We will initially work with this intuitive interpretation of the multinomial NB model and defer a formal derivation to Section 13.4.

How do we estimate the parameters $\hat{P}(c)$ and $\hat{P}(t_k|c)$? We first try the maximum likelihood estimate (MLE; Section 11.3.2, page 226), which is simply the relative frequency and corresponds to the most likely value of each parameter given the training data. For the priors this estimate is:

$$(13.5) \quad \hat{P}(c) = \frac{N_c}{N},$$

where N_c is the number of documents in class c and N is the total number of documents.

We estimate the conditional probability $\hat{P}(t|c)$ as the relative frequency of term t in documents belonging to class c :

$$(13.6) \quad \hat{P}(t|c) = \frac{T_{ct}}{\sum_{t' \in V} T_{ct'}},$$

where T_{ct} is the number of occurrences of t in training documents from class c , including multiple occurrences of a term in a document. We have made the *positional independence assumption* here, which we will discuss in more detail in the next section: T_{ct} is a count of occurrences in all positions k in the documents in the training set. Thus, we do not compute different estimates for different positions and, for example, if a word occurs twice in a document, in positions k_1 and k_2 , then $\hat{P}(t_{k_1}|c) = \hat{P}(t_{k_2}|c)$.

The problem with the MLE estimate is that it is zero for a term–class combination that did not occur in the training data. If the term WTO in the training data only occurred in *China* documents, then the MLE estimates for the other classes, for example *UK*, will be zero:

$$\hat{P}(\text{WTO}|\text{UK}) = 0.$$

Now, the one-sentence document *Britain is a member of the WTO* will get a conditional probability of zero for *UK* because we are multiplying the conditional probabilities for all terms in Equation (13.2). Clearly, the model should


```

TRAINMULTINOMIALNB( $\mathbf{C}, \mathbf{ID}$ )
1   $V \leftarrow \text{EXTRACTVOCABULARY}(\mathbf{ID})$ 
2   $N \leftarrow \text{COUNTDOCS}(\mathbf{ID})$ 
3  for each  $c \in \mathbf{C}$ 
4  do  $N_c \leftarrow \text{COUNTDOCSINCLASS}(\mathbf{ID}, c)$ 
5      $\text{prior}[c] \leftarrow N_c / N$ 
6      $\text{text}_c \leftarrow \text{CONCATENATETEXTOFALLDOCSINCLASS}(\mathbf{ID}, c)$ 
7     for each  $t \in V$ 
8     do  $T_{ct} \leftarrow \text{COUNTTOKENSOFTERM}(\text{text}_c, t)$ 
9     for each  $t \in V$ 
10    do  $\text{condprob}[t][c] \leftarrow \frac{T_{ct}+1}{\sum_{t'} (T_{ct'}+1)}$ 
11 return  $V, \text{prior}, \text{condprob}$ 

APPLYMULTINOMIALNB( $\mathbf{C}, V, \text{prior}, \text{condprob}, d$ )
1   $W \leftarrow \text{EXTRACTTOKENSFROMDOC}(V, d)$ 
2  for each  $c \in \mathbf{C}$ 
3  do  $\text{score}[c] \leftarrow \log \text{prior}[c]$ 
4     for each  $t \in W$ 
5     do  $\text{score}[c] += \log \text{condprob}[t][c]$ 
6  return  $\arg \max_{c \in \mathbf{C}} \text{score}[c]$ 

```

► **Figure 13.2** Naive Bayes algorithm (multinomial model): Training and testing.

assign a high probability to the *UK* class because the term *Britain* occurs. The problem is that the zero probability for *WTO* cannot be “conditioned away,” no matter how strong the evidence for the class *UK* from other features. The estimate is 0 because of *sparseness*: The training data are never large enough to represent the frequency of rare events adequately, for example, the frequency of *WTO* occurring in *UK* documents.

SPARSENESS

ADD-ONE SMOOTHING

To eliminate zeros, we use *add-one* or *Laplace smoothing*, which simply adds one to each count (cf. Section 11.3.2):

$$(13.7) \quad \hat{P}(t|c) = \frac{T_{ct} + 1}{\sum_{t' \in V} (T_{ct'} + 1)} = \frac{T_{ct} + 1}{(\sum_{t' \in V} T_{ct'}) + B},$$

where $B = |V|$ is the number of terms in the vocabulary. Add-one smoothing can be interpreted as a uniform prior (each term occurs once for each class) that is then updated as evidence from the training data comes in. Note that this is a prior probability for the occurrence of a *term* as opposed to the prior probability of a *class* which we estimate in Equation (13.5) on the document level.

► **Table 13.1** Data for parameter estimation examples.

	docID	words in document	in $c = \textit{China}$?
training set	1	Chinese Beijing Chinese	yes
	2	Chinese Chinese Shanghai	yes
	3	Chinese Macao	yes
	4	Tokyo Japan Chinese	no
test set	5	Chinese Chinese Chinese Tokyo Japan	?

► **Table 13.2** Training and test times for NB.

	mode	time complexity
training		$\Theta(\mathcal{D} L_{\text{ave}} + \mathcal{C} V)$
testing		$\Theta(L_a + \mathcal{C} M_a) = \Theta(\mathcal{C} M_a)$

We have now introduced all the elements we need for training and applying an NB classifier. The complete algorithm is described in Figure 13.2.



Example 13.1: For the example in Table 13.1, the multinomial parameters we need to classify the test document are the priors $\hat{P}(c) = 3/4$ and $\hat{P}(\bar{c}) = 1/4$ and the following conditional probabilities:

$$\begin{aligned}\hat{P}(\textit{Chinese}|c) &= (5+1)/(8+6) = 6/14 = 3/7 \\ \hat{P}(\textit{Tokyo}|c) = \hat{P}(\textit{Japan}|c) &= (0+1)/(8+6) = 1/14 \\ \hat{P}(\textit{Chinese}|\bar{c}) &= (1+1)/(3+6) = 2/9 \\ \hat{P}(\textit{Tokyo}|\bar{c}) = \hat{P}(\textit{Japan}|\bar{c}) &= (1+1)/(3+6) = 2/9\end{aligned}$$

The denominators are $(8+6)$ and $(3+6)$ because the lengths of $\textit{text}_c$ and $\textit{text}_{\bar{c}}$ are 8 and 3, respectively, and because the constant B in Equation (13.7) is 6 as the vocabulary consists of six terms.

We then get:

$$\begin{aligned}\hat{P}(c|d_5) &\propto 3/4 \cdot (3/7)^3 \cdot 1/14 \cdot 1/14 \approx 0.0003. \\ \hat{P}(\bar{c}|d_5) &\propto 1/4 \cdot (2/9)^3 \cdot 2/9 \cdot 2/9 \approx 0.0001.\end{aligned}$$

Thus, the classifier assigns the test document to $c = \textit{China}$. The reason for this classification decision is that the three occurrences of the positive indicator Chinese in d_5 outweigh the occurrences of the two negative indicators Japan and Tokyo.

What is the time complexity of NB? The complexity of computing the parameters is $\Theta(|\mathcal{C}||V|)$ because the set of parameters consists of $|\mathcal{C}||V|$ conditional probabilities and $|\mathcal{C}|$ priors. The preprocessing necessary for computing the parameters (extracting the vocabulary, counting terms, etc.) can be done in one pass through the training data. The time complexity of this

component is therefore $\Theta(|\mathbb{D}|L_{\text{ave}})$, where $|\mathbb{D}|$ is the number of documents and L_{ave} is the average length of a document.

We use $\Theta(|\mathbb{D}|L_{\text{ave}})$ as a notation for $\Theta(T)$ here, where T is the length of the training collection. This is nonstandard; $\Theta(\cdot)$ is not defined for an average. We prefer expressing the time complexity in terms of $\mathbb{D}$ and L_{ave} because these are the primary statistics used to characterize training collections.

The time complexity of APPLYMULTINOMIALNB in Figure 13.2 is $\Theta(|\mathbb{C}|L_a)$. L_a and M_a are the numbers of tokens and types, respectively, in the test document. APPLYMULTINOMIALNB can be modified to be $\Theta(L_a + |\mathbb{C}|M_a)$ (Exercise 13.8). Finally, assuming that the length of test documents is bounded, $\Theta(L_a + |\mathbb{C}|M_a) = \Theta(|\mathbb{C}|M_a)$ because $L_a < b|\mathbb{C}|M_a$ for a fixed constant b .²

Table 13.2 summarizes the time complexities. In general, we have $|\mathbb{C}||V| < |\mathbb{D}|L_{\text{ave}}$, so both training and testing complexity are linear in the time it takes to scan the data. Because we have to look at the data at least once, NB can be said to have optimal time complexity. Its efficiency is one reason why NB is a popular text classification method.

13.2.1 Relation to multinomial unigram language model

The multinomial NB model is formally identical to the multinomial unigram language model (Section 12.2.1, page 242). In particular, Equation (13.2) is a special case of Equation (12.12) from page 243, which we repeat here for $\lambda = 1$:

$$(13.8) \quad P(d|q) \propto P(d) \prod_{t \in q} P(t|M_d).$$

The document d in text classification (Equation (13.2)) takes the role of the query in language modeling (Equation (13.8)) and the classes c in text classification take the role of the documents d in language modeling. We used Equation (13.8) to rank documents according to the probability that they are relevant to the query q . In NB classification, we are usually only interested in the top-ranked class.

We also used MLE estimates in Section 12.2.2 (page 243) and encountered the problem of zero estimates owing to sparse data (page 244); but instead of add-one smoothing, we used a mixture of two distributions to address the problem there. Add-one smoothing is closely related to add- $\frac{1}{2}$ smoothing in Section 11.3.4 (page 228).

?

Exercise 13.1

Why is $|\mathbb{C}||V| < |\mathbb{D}|L_{\text{ave}}$ in Table 13.2 expected to hold for most text collections?

2. Our assumption here is that the length of test documents is bounded. L_a would exceed $b|\mathbb{C}|M_a$ for extremely long test documents.

```

TRAINBERNOULLINB( $\mathbf{C}, \mathbf{ID}$ )
1  $V \leftarrow \text{EXTRACTVOCABULARY}(\mathbf{ID})$ 
2  $N \leftarrow \text{COUNTDOCS}(\mathbf{ID})$ 
3 for each  $c \in \mathbf{C}$ 
4   do  $N_c \leftarrow \text{COUNTDOCSINCLASS}(\mathbf{ID}, c)$ 
5      $\text{prior}[c] \leftarrow N_c / N$ 
6     for each  $t \in V$ 
7       do  $N_{ct} \leftarrow \text{COUNTDOCSINCLASSCONTAININGTERM}(\mathbf{ID}, c, t)$ 
8          $\text{condprob}[t][c] \leftarrow (N_{ct} + 1) / (N_c + 2)$ 
9 return  $V, \text{prior}, \text{condprob}$ 

APPLYBERNOULLINB( $\mathbf{C}, V, \text{prior}, \text{condprob}, d$ )
1  $V_d \leftarrow \text{EXTRACTTERMSFROMDOC}(V, d)$ 
2 for each  $c \in \mathbf{C}$ 
3   do  $\text{score}[c] \leftarrow \log \text{prior}[c]$ 
4     for each  $t \in V$ 
5       do if  $t \in V_d$ 
6         then  $\text{score}[c] += \log \text{condprob}[t][c]$ 
7         else  $\text{score}[c] += \log(1 - \text{condprob}[t][c])$ 
8 return  $\arg \max_{c \in \mathbf{C}} \text{score}[c]$ 

```

► **Figure 13.3** NB algorithm (Bernoulli model): Training and testing. The add-one smoothing in Line 8 (top) is in analogy to Equation (13.7) with $B = 2$.

13.3 The Bernoulli model

There are two different ways we can set up an NB classifier. The model we introduced in the previous section is the multinomial model. It generates one term from the vocabulary in each position of the document, where we assume a generative model that will be discussed in more detail in Section 13.4 (see also page 237).

BERNOULLI MODEL An alternative to the multinomial model is the *multivariate Bernoulli model* or *Bernoulli model*. It is equivalent to the binary independence model of Section 11.3 (page 222), which generates an indicator for each term of the vocabulary, either 1 indicating presence of the term in the document or 0 indicating absence. Figure 13.3 presents training and testing algorithms for the Bernoulli model. The Bernoulli model has the same time complexity as the multinomial model.

The different generation models imply different estimation strategies and different classification rules. The Bernoulli model estimates $\hat{P}(t|c)$ as the *fraction of documents* of class c that contain term t (Figure 13.3, TRAINBERNOULLI-